Rapport du Projet P2I2
KLEPS

Norman ASSING, Clément COUCHEVELLOU
Titouan COUVRAT–PAILLE, Khalyl SEDEKI

6 janvier 2022

Table des matières

1 Introduction au sujet **2**
1.1 Contexte du projet 2
1.2 Organisation du document 2
1.3 Remerciements 3
1.4 Précisions légales 3

2 Etat de l'art **4**
2.1 Civic Tech et applications de débat 4
2.2 Bilan et concept de l'application 4
2.3 Compréhension d'algorithmes 5

3 Gestion de projet **8**
3.1 Equipe de projet 8
3.2 Analyse du projet 9
3.3 Organisation 12
3.4 Outils de travail 14

4 Conception de l'application **16**
4.1 Partie Python 16
4.2 Partie Base de données 17
4.3 Partie Web 19
4.4 Partie Tests 20

5 Bilan de projet **22**
5.1 Bilan sur l'avancement du projet 22
5.2 Bilan global du projet 23
5.3 Bilan du projet par membre 25

Chapitre 1

Introduction au sujet

Contents

1.1 Contexte du projet 2
1.2 Organisation du document 2
1.3 Remerciements 3
1.4 Précisions légales 3

1.1 Contexte du projet

Ce projet a été réalisé dans le cadre du module P2I2 (Projet Pluridisciplinaire d'Informatique Intégrative) de la première année du cycle ingénieur sous statut étudiant de TÉLÉCOM Nancy.

L'objectif est de concevoir une application Web s'inscrivant dans le cadre de la Civic Tech, c'est-à-dire une application capable de faciliter la démocratie participative locale.

Le travail s'est décomposé en trois parties : la recherche de fonctionnalités, la réalisation de l'application, et la gestion du projet en équipe.

1.2 Organisation du document

Dans le chapitre 2, nous analysons le domaine de la Civic Tech, en listant les projets existants et en étudiant les fonctionnalités proposées par ces projets. Nous faisons de même pour le cas particulier des plateformes de débat.

Dans le chapitre 3, nous présentons les éléments ainsi que les outils de gestion de projet que nous avons utilisés.

Dans le chapitre 4, nous listons et expliquons les fonctionnalités que nous avons réussi à implémenter, ainsi que les tests et les mesures de performance.

Dans le chapitre 5, nous réalisons les bilans, personnels et globaux, du projet.

1.3 Remerciements

La structure globale de ce rapport a été basée sur celle du rapport du projet *Jouons à Cache-Cache* de CHANTELOUP Marie-Astrid, RESZETKO Clément et SEPE Benjamin, présenté lors du tutorat de rédaction de rapport. Cela représente un gain de temps considérable, surtout en fin de projet. L'équipe remercie grandement ces personnes pour leur aide.

1.4 Précisions légales

Ce projet n'est pas destiné à un usage commercial, ainsi, les images présentes, notamment les images de test, ne sont pas destinées à la publication et ne sont pas toutes libres de droit.

Cependant, le caractère strictement scolaire de ce projet nous autorise à les inclure en accord avec :

- Code civil : articles 7 à 15, article 9 : respect de la vie privée
- Code pénal : articles 226-1 à 226-7 : atteinte à la vie privée
- Code de procédure civil : articles 484 à 492-1 : procédure de référé
- Loi n78-17 du 6 janvier 1978 : Informatique et libertés, Article 38

Chapitre 2

Etat de l'art

Contents

2.1 Civic Tech et applications de débat 4
2.2 Bilan et concept de l'application 4
2.3 Compréhension d'algorithmes 5
2.3.1 Introduction 5
2.3.2 HTML 5
2.3.3 CSS 5
2.3.4 PHP 6
2.3.5 Flask 6
2.3.6 JavaScript 6
2.3.7 Messages audio 7
2.3.8 Tri 7
2.3.9 RegEx 7
2.3.10 Système de gestion de base de données 7

2.1 Civic Tech et applications de débat

– Voir fichiers annexes –

2.2 Bilan et concept de l'application

En observant tous ces projets, soit pas vraiment en rapport avec la Civic Tech, soit n'ayant aucune communauté, il semblerait que les deux plus gros problèmes de ce domaine sont le manque de marketing et des concepts dont on fait vite le tour. Mais il y a cependant des points plus positifs à relever, comme

le projet de DevFoundry qui utilise autre chose que des simples messages textes, ou Agora, qui permet de citer d'autres messages qui peut faire avancer le débat. Cela nous a encouragé à nous appuyer sur des messages audios, et à utiliser une structure similaire à celle d'un réseau social. Enfin, il nous semble intéressant de vouloir adresser notre projet aux communes, afin de leur déléguer la partie Marketing.

2.3 Compréhension d'algorithmes

2.3.1 Introduction

Afin de créer notre application Web, nous avons utilisé du HTML pour le fond et du CSS pour la forme. Cependant pour obtenir une application plus interactive et générée procéduralement, mais aussi afin de permettre aux utilisateurs de communiquer entre eux et d'échanger des informations, nous avons aussi utilisé du Python, du SQL et un peu de JavaScript.

2.3.2 HTML

HTML est un langage qui permet de générer des pages statiques et de récupérer des données via des "form", l'esthétique de ces pages est gérée par CSS. Le problème étant que pour réaliser notre site de débat il nous faut générer des pages dynamiquement, de manière à permettre aux utilisateurs d'interagir entre eux. Nous avons donc dû trouver un moyen de générer des pages HTML dynamiquement, nos principales pistes étant PHP et Flask.

2.3.3 CSS

Pour la mise en page de notre site nous avons utilisé le langage CSS. Après des recherches sur les différents types de CSS tels que Flex, Bootstrap ou Grid, Flex nous paraissait être la meilleure option. Bootstrap étant rapide et probablement plus poussé que notre niveau en CSS actuel, nous avons écarté l'option car pas assez transparente. Certes il permet des rendus plus esthétiques dans certains cas, mais le fait de réutiliser du code sans se pencher dessus était un problème éthique, le souhait de produire quelque chose de nos mains uniquement était un objectif tacite. Ainsi, Flex paraissait la meilleure solution, et la majorité du CSS utilise Flex. L'apparence du site étant un point important pour les utilisateurs, nous avons investi du temps sur ce volet du projet. Le peu de JavaScript utilisé est seulement pour la partie

audio, nous n'avions pas le temps de se former sur le langage pour l'utiliser dans la mise en page du site

2.3.4 PHP

PHP étant le langage de programmation le plus utilisé pour coder des sites web, nous nous sommes naturellement tourné vers lui dans un premier temps, il correspondait exactement à ce dont nous avions besoin, étant donné qu'il permet de créer des sites web dynamique, parfait créer notre site de débat. Le principal avantage de PHP est qu'il est assez facile à prendre en main.
Cependant il n'était pas possible de l'utiliser tout en respectant notre cahier des charges, nous avons donc abandonné PHP, jusqu'au moment où nous avons dû programmer la récupération des messages audio via JavaScript, car il se trouve que ce genre de fonctionnalité est souvent géré grâce au duo JavaScript/PHP, cependant nous avons réussi à nous passer de PHP, au profit de Flask.

2.3.5 Flask

Flask est un module python qui permet de générer dynamiquement des pages HTML, ce qui est nécessaire pour créer notre application de débats, car il nous faut ajouter dynamiquement débats, commentaires, jetons... Le fonctionnement de base est assez simple, il permet à partir de fonctions python de générer des pages HTML à partir de modèles (templates). Il est possible de transmettre aux pages HTML des données à partir de variables python, ces données sont gérés avec Jinja, afin de générer du code HTML dynamiquement, de manière similaire au fonctionnement de PHP. Flask offre aussi la possibilité de créer des "sessions" qui permettent de stocker des informations sur l'utilisateur et de les transmettre entre les différentes pages, afin qu'il n'ait à se connecter qu'une seule fois. Flask permet également de gérer la récupération de fichier via form HTML et permet d'interagir avec sous python.

2.3.6 JavaScript

JavaScript est un langage de programmation web permettant de réaliser des énormement de chose, allant de simple implémentation esthétique, à des fonctionnalités beaucoup plus poussées comme l'enregistrement audio que nous utilisons, après de longues recherches, JavaScript nous a semblé être le meilleur, voir peut-être même le seul moyen de réaliser l'enregistrement des messages audio.

2.3.7 Messages audio

Pour commencer il nous faut enregistrer le message audio, pour cela nous avons décidé d'utiliser JavaScript, qui était le meilleur, voir le seul moyen d'enregistrer des messages audio depuis une page html. JavaScript nous permet également de proposer à l'utilisateur de télécharger son son message audio. C'est à ce moment-là qu'il faut repasser sur HTML, afin de créer un "form" permettant à l'utilisateur d'envoyer le message audio qu'il vient de télécharger. Les données du "form" sont ensuite envoyées vers une route Flask pour être traitées sous python, ce qui permet de les déplacer dans le serveur et de les enregistrer dans la base de données.

Pour une optimisation du stockage, ainsi que pour une compatibilité maximale entre les navigateurs, l'utilisation du logiciel ffmpeg est essentielle à la fois pour la conversion et la compression des messages audio. C'est à la base un fichier exécutable qu'on utilise avec une console, mais il existe des bibliothèques Python qui ont les mêmes fonctionnalités. On peut citer pydub et ffmpeg-python notamment, qui se basent tous les deux sur ffmpeg.

2.3.8 Tri

Il existe pléthore de tri en python, cependant notre objectif est de réaliser une application web, il nous faut donc un tri le plus efficace possible, les utilisateurs ne doivent pas attendre entre chaque page, nous avons donc choisi d'utiliser un algorithme de Tri Rapide ou encore Quick Sort en anglais. Sa complexité pour une liste de n éléments étant de l'ordre de n.log(n) c'est un des tri le plus efficace que nous pouvions utiliser.

2.3.9 RegEx

Pour ajouter des restrictions dans le login sur le nom d'utilisateur ou l'email, nous avons utilisé les Regex couplés à l'attribut "pattern" de html pour être sur qu'un email soit au bon format ou qu'un username ne contient pas d'espace.

2.3.10 Système de gestion de base de données

Pour stocker nos données il nous fallait un système de base de données, après avoir essayé MySQL, PostgreSQL et SQLite 3, nous avons décidé de choisir SQLite3, pour sa simplicité d'utilisation et de génération de fichiers .db.

Chapitre 3

Gestion de projet

Contents

3.1 Equipe de projet 8
3.2 Analyse du projet 9
3.2.1 Définition du projet 9
3.2.2 Définition des lots 9
3.2.3 Matrice SWOT 11
3.3 Organisation 12
3.3.1 Jalonnement 12
3.3.2 Diagramme de Gantt 12
3.3.3 Lot 3.3 12
3.3.4 Répartition des tâches 12
3.4 Outils de travail 14
3.4.1 IDE 14
3.4.2 Outils de communication 15
3.4.3 Outils de travail collaboratif 15
3.4.4 Outils de mise en forme 15

3.1 Equipe de projet

L'équipe se compose de quatre étudiants en première année :
— ASSING Norman
— COUCHEVELLOU Clément
— COUVRAT–PAILLE Titouan
— SEDEKI Khalyl

Norman, en tant qu'initiateur du projet, est désigné comme chef de projet. Ce sera lui qui suivra l'avancement du projet.

Durant la période scolaire, l'équipe se réunissait une fois par semaine, sans créneau fixe particulier. Pendant les vacances, les réunions s'effectuaient tous les deux ou trois jours, en général vers 22h.

Les éléments de programmation ont été créés, partagés et utilisés à partir du GitLab fourni par l'école. Le rapport a été rédigé dans un premier temps sur Google Drive, puis a été porté sur LateX.

3.2 Analyse du projet

3.2.1 Définition du projet

Les objectifs de ce projet sont le développement d'une application web, dans le domaine de la Civic Tech, permettant à des organismes publiques de mieux comprendre les attentes des citoyens, mais également de découvrir de nouvelles solutions innovantes à des problèmes locaux.
Le livrable principal est la création d'une application web de débats, via des messages vocaux, organisés grâce à un système de votes et de commentaires.

3.2.2 Définition des lots

Les lots ont été définis à l'aide de la méthode **SMART** : "**S**pécifique, **M**esurable, **A**tteignable, **R**éaliste, délimité dans le **T**emps ". Notons tout de même que le dernier critère n'a pas toujours été respecté, étant donné nos connaissances en informatique toujours en développement.
Le schéma ci-dessous a été réalisé via diagrams.net en fin de projet, afin de comprendre rapidement la structure globale du site, en se basant sur les routes Flask. En début de projet, et afin de déterminer les lots de la manière la plus exhaustive possible, le schéma ci-dessous a été réalisé via diagrams.net également. Il liste toutes les interactions possibles entre les utilisateurs. La liste des lots est détaillée dans la matrice RACI du chapitre 3.3.4 .
Notons cependant que cette méthode a été très utile au départ, mais finalement, on peut lui faire quelques critiques. En effet, certaines flèches sont un peu superficielles et il y a souvent un déséquilibre entre la quantité de travail de chaque lot. De plus, pour le lot 3.3, qui s'appuie moins sur du front-end que les autres, il était difficile d'intégrer cette partie au schéma, et il a donc fallu renseigner les lots dans la matrice au fur et à mesure qu'on ajoutait des fonctionnalités.

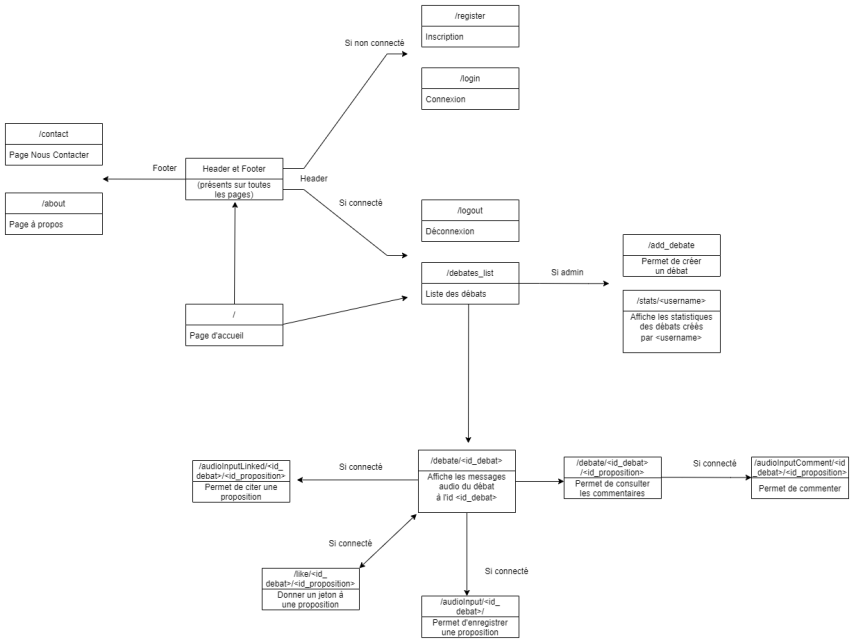

FIGURE 3.1 – Schéma des routes du projet

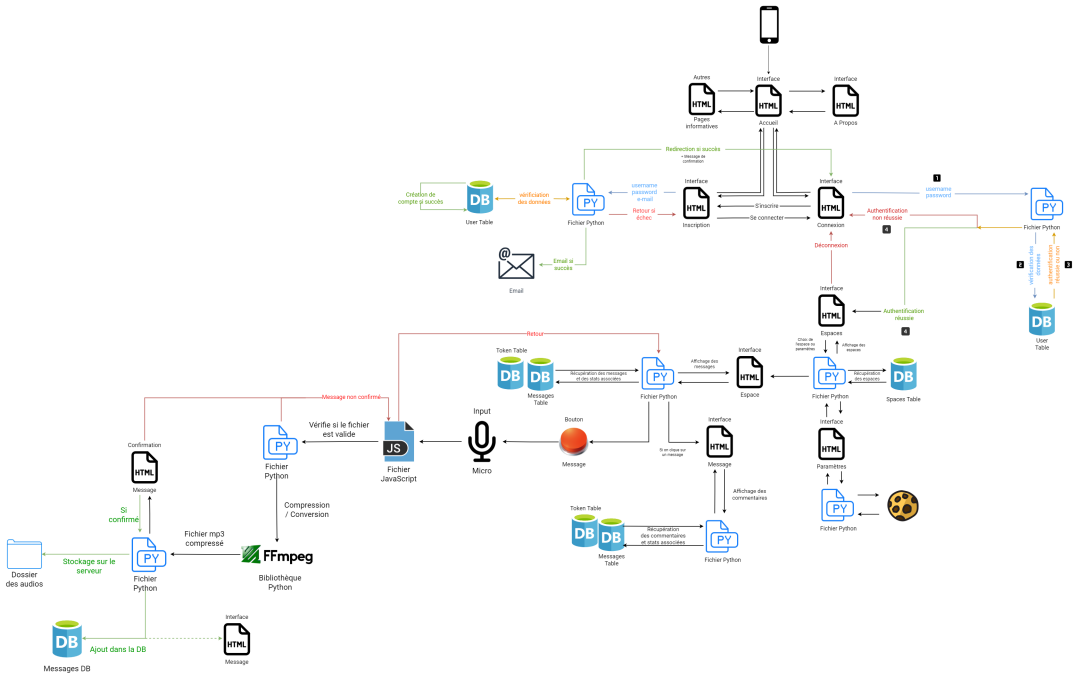

FIGURE 3.2 – Schéma général du projet

3.2.3 Matrice SWOT

Voici les forces, faiblesses, opportunités et menaces qui se sont ou auraient pu se présenter à nous durant ce projet :

FIGURE 3.3 – Matrice SWOT

3.3 Organisation

3.3.1 Jalonnement

Le projet se déroule entre la fin du mois d'octobre et jusqu'à début janvier. Afin de développer l'application au mieux en s'assurant de rester dans les délais, le groupe a décidé de cinq jalons, dont les dates ont été fixées fin novembre. La répartition des lots s'est faite au fur et à mesure et en fonction de la disponibilité de chacun.

3.3.2 Diagramme de Gantt

Le diagramme de Gantt ci-dessous a servi de calendrier pour ces jalons, et représente également les écarts entre le temps prévu pour chaque tâche et la réalité.

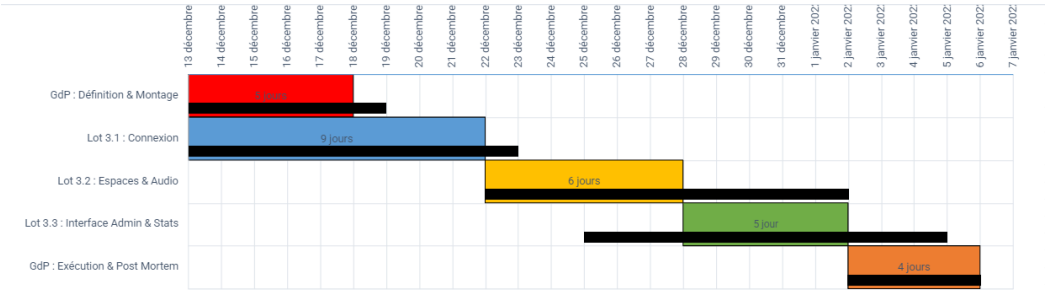

FIGURE 3.4 – Diagramme de Gantt

3.3.3 Lot 3.3

Avant les vacances, tous les lots et sous-lots avaient été déterminés sauf le lot 3.3, constitué des systèmes de vote et de mise en avant des propositions. Les membres ayant des avis partagés quant à la manière d'implémenter ces fonctionnalités, la définition de ce lot se fera plus tard durant le processus de développement.

3.3.4 Répartition des tâches

Pour mieux visualiser la répartition des tâches, l'équipe a utilisé Google Sheet pour faire une matrice **RACI** : **R**esponsible , **A**ccountable , **C**onsulted, **I**nformed. Comme tout le groupe était constamment informé de l'avancement de chacun, que personne n'avait assez de connaissance pour avoir le grade

d'expert, et que le contrôle des lots se faisait principalement par tous en réunion, la matrice RACI est principalement remplie de R.

Lot	Description	Fini	Norman	Clément	Titouan	Khalyl
1.1	GdP - Définition					
1.1.1	Charte du Projet		A	R		R
1.1.2	Cahier des Charges		A	R	A	
1.1.3	SWOT		A	A	R	A
1.2	GdP - Montage					
1.2.1	Diagramme de Gantt		R	A	A	A
1.2.2	Matrice RACI		R	A	A	A
1.3	Comptes Rendus		RA	RA	RA	RA

FIGURE 3.5 – Gestion de Projet

Lot	Description	Fini	Norman	Clément	Titouan	Khalyl
2.1	Etat de l'Art					
2.1.1	Etat de l'Art - Civic Tech			A		R
2.1.2	Etat de l'Art - Apps de débats		R	A		

FIGURE 3.6 – Etat de l'art

Lot	Description	Fini	Norman	Clément	Titouan	Khalyl
3.1.1	Interface Accueil		A	A	R	A
3.1.2	Interface A Propos		A	A	R	A
3.1.3	Lien Accueil - A propos		R	A	A	A
3.1.4	Interface Nous Contacter		R	A	A	A
3.1.5	Lien Accueil - Nous Contacter		R	A	A	A
3.1.6	Interface Inscription		R	A	A	A
3.1.7	Lien Accueil - Inscription		R	A	A	A
3.1.8	Inscription : Envoi des infos de login (username,password,email)		A	R	A	A
3.1.8.1	Vérifications des données via Python		R	A	A	A
3.1.8.2	Vérifications des données via la BD		A	A	A	R
3.1.8.3	Si échec : Retour vers Interface Inscription + popup		A	R	A	A
3.1.8.3.1	Si succès : Création du compte dans la table		R	R	A	A
3.1.8.3.2	Si succès : Email de confirmation (+)					
3.1.8.3.3	Si succès : Redirection vers Interface Connexion + popup		R	R	A	A
3.1.9	Interface Connexion		R	A	R	A
3.1.10	Lien Accueil - Connexion		R	A	A	A
3.1.11	Lien Inscription - Connexion (se connecter ou s'inscrire)		R	A	A	A
3.1.12	Connexion : Envoi des infos de login (username,password)		R	R	A	A
3.1.12.1	Vérifications des données via Python		A	R	A	A
3.1.12.2	Vérifications des données via la BD		A	A	A	R
3.1.12.3	Si échec : Retour vers Interface Connexion + popup		A	R	A	A
3.1.12.4	Si succès : Redirection vers Interface Espaces		R	A	A	A

FIGURE 3.7 – Lot 3.1 : Partie Connexion

Lot	Description	Fini	Norman	Clément	Titouan	Khalyl
3.2.1	Interface Espaces		RA	RA	RA	A
3.2.2	Déconnexion		A	R	A	A
3.2.3	Récupération du nom des espaces via la BDD		R	A	A	A
3.2.4	Génération de la page Web Espaces à partir des données		R	A	R	A
3.2.5	Interface Paramètres/Préférences					
3.2.6	Lien Espaces-Paramètres					
3.2.7	Modification des cookies		RA	RA	RA	A
3.2.8	Interface Espaces		RA	RA	RA	A
3.2.9	Récupération des messages via la BDD		R	R	A	A
3.2.10	Génération de la page Web Espace à partir des données		R	A	A	A
3.2.11	Interface Envoyer un message		RA	RA	A	A
3.2.12	Enregistrement du message		A	R	A	A
3.2.13	Stockage temporaire du message		Pas nécessaire			
3.2.14	Vérification des données via Python		A	R	A	A
3.2.15	Si échec : Retour vers Interface Envoyer un message + popup		A	R	A	A
3.2.16.1	Si succès : Compression/Convertion (FFMPEG)					
3.2.16.2	Si succès : Redirection vers Interface Confirmation Message		A	R	A	A
3.2.17	Interface Confirmation Message		Pas nécessaire			
3.2.18	Faire écouter le message à l'utilisateur		Pas nécessaire			
3.2.19.1	Si annulation : Effacer le message		Pas nécessaire			
3.2.19.2	Si annulation : Retour vers Interface Envoyer un message + Popup		A	R	A	A
3.2.20.1	Si confirmation : Stockage permanent du message		A	R	A	A
3.2.20.2	Si confirmation : Ajout dans la BDD		A	R	A	A
3.2.20.3	Si confirmation : Redirection vers Interface Message		A	R	A	A
3.2.21	Tables de la base de données		A	RA	A	R

FIGURE 3.8 – Lot 3.2 : Partie Espaces et Messages Audio

Lot	Description		Norman	Clément	Titouan	Khalyl
3.3.1	Statistiques					
3.3.1.1.1	Voter avec des jetons		R	A	A	A
3.3.1.1.2	Récupérer nombre de votes		R	A	A	R
3.3.1.1.3	Limiter le nombre de votes					
3.3.1.2.1	Commenter		A	R	A	A
3.3.1.2.2	Récupérer nombre de commentaires		A	A	A	R
3.3.1.3	Citer		R	R	A	A
3.3.1.4.1	Algorithme de recommandation		R	A	A	A
3.3.1.4.2	Différents tri (récence, popularité...)		A	R	A	A
3.3.1.5	Page pour une proposition					
3.3.1.6.1	Interface Proposition		R	A	R	A
3.3.1.6.2	Récupération des commentaires		A	R	A	A
3.3.2	Interface Admin					
3.3.2.1	Interface Créer Débats		RA	RA	RA	A
3.3.2.2	Interface Stat pour chaque débat		A	A	A	R
3.3.2.3	Arrêter un débat					
3.3.2.4	Fonction Python pour créer des admins sans avoir à arrêter le serveur		R	R	A	A
3.3.2.5	Refonte du CSS du site		A	A	R	A

FIGURE 3.9 – Lot 3.3 : Partie Interface Admin et Statistiques

3.4 Outils de travail

3.4.1 IDE

L'ensemble du groupe a travaillé sur Visual Studio Code.

3.4.2 Outils de communication

Les réunions se déroulaient généralement via Discord, mais étant donné les agendas de chacun, il était difficile de tout faire rentrer lors de ces réunions, ainsi beaucoup de décisions et de répartition des tâches ont été faites via la conversation Messenger du projet. Certains comptes rendus, inachevés, ont pu être complétés en reprenant les messages de la conversation.

3.4.3 Outils de travail collaboratif

Pour synchroniser nos travaux, nous avons utilisé le GitLab de l'école pour tout l'aspect programmation. L'aspect gestion de projet a lui plutôt été géré sur Google Drive, puis déposé sur le GitLab. Nous avons utilisé le logiciel Trello pour gérer nos to-do listes. Il nous permis de suivre l'avancée des lots du projet de manière synchronisée et efficace.

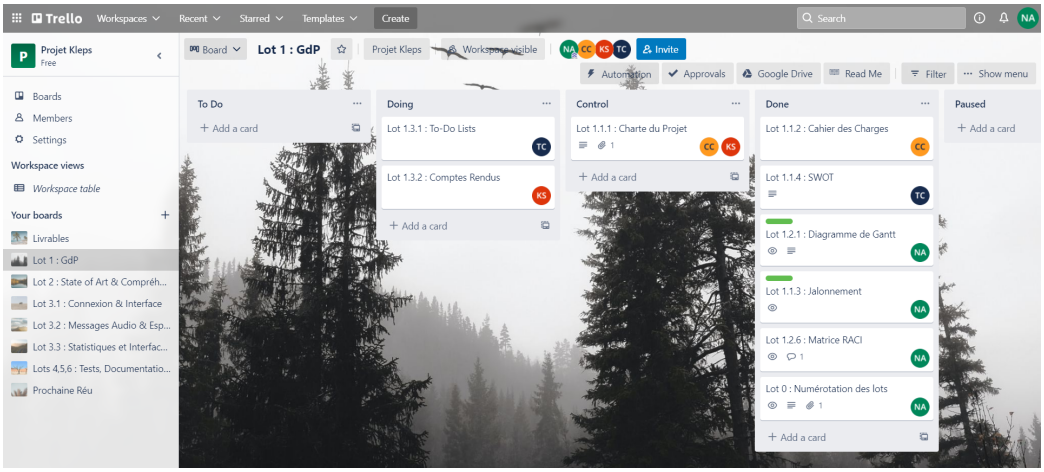

FIGURE 3.10 – Capture d'écran de Trello

3.4.4 Outils de mise en forme

L'équipe a utilisé diverses solutions pour illustrer ses idées, notamment Canva pour les présentations et la matrice SWOT, diagrams.net pour le schéma de fonctionnement de l'application, ainsi qu'Excel pour le diagramme de Gantt.

Chapitre 4

Conception de l'application

Contents

4.1 Partie Python 16
4.1.1 Comptes utilisateurs 16
4.1.2 Tri par popularité 17
4.1.3 Administrateurs 17
4.2 Partie Base de données 17
4.2.1 Structure de la base de données 17
4.2.2 Stockage des messages audio 18
4.2.3 Administrateurs 18
4.2.4 Statistiques 18
4.3 Partie Web 19
4.3.1 Interface 19
4.3.2 Messages audio 19
4.3.3 Espaces de débats 19
4.3.4 Interface admin 19
4.3.5 Statistiques 20
4.4 Partie Tests 20
4.4.1 Performances 20
4.4.2 Correction de l'algorithme 20

4.1 Partie Python

4.1.1 Comptes utilisateurs

Bien que la majorité des fonctionnalités liées aux comptes utilisateurs soient gérées par Flask et SQL, nous utilisons tout de même python afin

de vérifier au moment de l'inscription que personne ne puisse utiliser un email ou un nom d'utilisateur déjà présent dans la Table 'userInfo'. Nous avons également utilisé une variable '*is_logged_in*' permettant de vérifier qu'un utilisateur est bien connecté avant de rejoindre certaines pages de l'application. Le fonctionnement général de tout le système des utilisateurs est géré par Flask grâce au système de "session" permettant de passer des informations liées à l'utilisateur d'une page à une autre sans que l'utilisateur n'ait quoi que ce soit à faire.

4.1.2 Tri par popularité

Par défaut, les messages dans un débat sont triés par 'popularité', ce qui correspond à un tri que nous avons imaginé, utilisant la formule suivante :

$$\frac{(3 \times Jetons + 2 \times Commentaires + Nombre\ de\ messages\ cits)}{\sqrt{Heures\ depuis\ creation}}$$

Ensuite le tri est réalisé grâce à un algorithme de Tri Rapide, triant la liste des scores de popularité et en parallèle celle des id des utilisateurs.

4.1.3 Administrateurs

Nous avons créé un fichier Python permettant d'ajouter rapidement des administrateurs. Le rôle d'administrateur a un intérêt majeur, qui est celui de pouvoir créer un débat, sur la page qui liste l'ensemble des débats, les administrateurs ont un bouton en plus le permettant, il leur faut ensuite choisir un nom et une description pour leur débat. La vérification du rôle se fait à partir d'une fonction python interagissant avec notre base de données, puis une page web.

4.2 Partie Base de données

4.2.1 Structure de la base de données

Notre base de données est composée de cinq tables, 'userInfo' composé des informations liées à chaque utilisateur, 'debates' contenant les caractéristiques de chaques débats, 'messages' contenant les informations liées aux propositions mais aussi aux commentaires, 'tokens' la table utilisé pour gérer le système de jetons et enfin '*spaces_users*' une table permettant de créer des débats privées nécessitant une invitation, cependant cette partie n'est pas fonctionnelle, faute de temps.

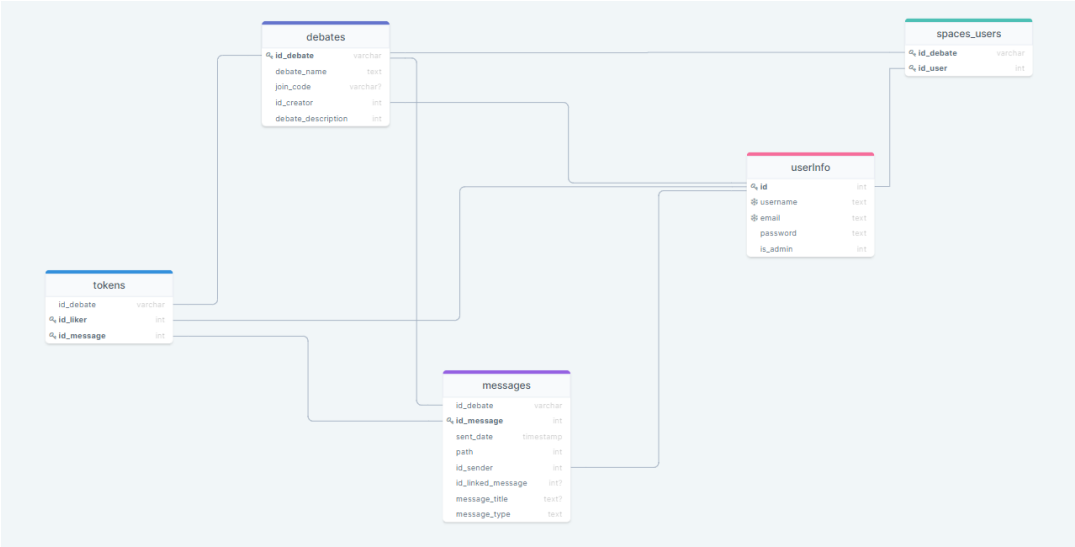

FIGURE 4.1 – Schéma des bases de données

4.2.2 Stockage des messages audio

Les messages audio sont récupérés depuis une page web et stockés dans un fichier sur le serveur, la base de données contient uniquement les chemins permettant de lier les données liées à un message audio au fichier correspondant.

4.2.3 Administrateurs

Les administrateurs de l'application sont déterminés par un attribut dans la table 'userInfo' qui vaut 1 si l'utilisateur est administrateur et 0 sinon, la vérification se fait donc forcément via une requête SQL.

4.2.4 Statistiques

Utilisation de requêtes avec plusieurs conditions pour différencier les commentaires des propositions et ainsi pouvoir obtenir le nombre de commentaires pour chaque proposition et donc ensuite pour chaque débat. Utilisation d'une table dédiée au Jetons, permettant de tracer chaque jetons utilisé et donc pouvoir ensuite les compter, pour un débat, une proposition ou même un utilisateur, malheureusement nous n'avons pas eu le temps d'implémenter la page de statistiques liée à chaque utilisateur.

4.3 Partie Web

4.3.1 Interface

L'interface est divisée verticalement, on accède d'abord à la liste des débats, puis on choisit un débat, on accède alors à la liste des propositions postées dans ce débat et si l'on décide de cliquer sur une proposition on accède alors à la liste des commentaires postés à propos de cette proposition. Nous avons accordé beaucoup d'importance à l'interface. Pour l'équipe le projet n'était pas seulement un rendu académique à rendre, mais un rendu ayant une réelle application dans le monde. Il était donc important de penser aux utilisateurs et ainsi d'avoir une interface soignée.

4.3.2 Messages audio

L'enregistrement des messages audio sur une page web est géré via JavaScript, seule solution trouvée par le groupe, le script est donc probablement très peu optimisé. Les messages audio sont ensuite récupérés sous forme de fichier .weba via un form de type 'POST' permettant le transfert des fichiers, la suite est gérée via Flask et Python.

4.3.3 Espaces de débats

Les espaces de débats sont gérés comme des listes des propositions triées par défaut selon le tri explicité en 4.1.2, mais il est également possible de choisir un tri favorisant uniquement le fait qu'un message soit récent, ce système de choix est géré très facilement, chaque tri correspond à un templates et le bouton de tri permet simplement de passer d'un template à l'autre. Chaque proposition a également une interface propre à laquelle on peut accéder en cliquant sur commentaires, propositions liées, jetons ou date. Cette interface permet de voir les commentaires liés à la proposition en question et d'en poster un.

4.3.4 Interface admin

La seule différence entre l'interface utilisateur et l'interface administrateur est l'ajout d'un bouton 'Créer un débat' sur la page qui liste les débats, la vérification se fait dans le fichier html via Jinja, à partir d'un booléen récupéré en amont grâce à une fonction python.

4.3.5 Statistiques

Les nombre de commentaires et jetons sont affichés sous chaque débat sur la page Espaces de débats, les nombres de commentaires, jetons et propositions liées à chaque propositions sont également affichés sous chaque proposition.

4.4 Partie Tests

4.4.1 Performances

Taille de la liste	Temps moyen d'exécution en seconde
100	0.0002989768981933594
1000	0.004224634170532227
10000	0.05303082466125488
100000	1.1606621503829957
1000000	15.647940135002136

Table 4.1 – Performance tri rapide.

La mesure des performances du tri rapide que nous utilisons pour trier les propositions par popularité à été réalisé sur un fichier python annexe présent dans le git sous le nom 'Performance.py', en utilisant les modules python time pour la mesure du temps d'exécution et random afin de générer des listes de test à trier.
Le script python renvoie le temps moyen de tri sur 10 exécutions, pour différentes tailles de listes composées de valeurs allant de 0 à 1000 avec deux chiffres après la virgule, les résultats obtenus pour une exécution du script sont contenus dans la Table 4.1 ci dessus.
Le tri est donc très rapide même sur des quantités de données disproportionnées, comparé à celles que notre application doit gérer.

4.4.2 Correction de l'algorithme

Notre algorithme de tri est rapide c'est une première chose, mais pour être sûr qu'il renvoie un résultat correct, nous avons décidé d'utiliser pytest, comme conseillé. Ainsi nous avons comparé le résultat obtenu avec notre tri et celui obtenu grâce au tri intégré à python 'list.sort()', nous avons répété le test avec 100 listes différentes obtenues aléatoirement de la même manière que pour le test des performances, avec cette fois ci un nombre de valeurs

fixé à 10000. Le script utilisé est '*test_sort.py*', il est placé dans le dossier annexes python, dans le git du projet. Le résultat obtenue avec pytest est le suivant : Le test est donc validé, notre tri est correct sur les 100 tests passés.

```
C:\Users\cleme\Desktop\master\project1-grp_e18\Script_python_annexes>python -m pytest
============================================= test session starts =============================================
platform win32 -- Python 3.9.9, pytest-6.2.5, py-1.11.0, pluggy-1.0.0
rootdir: C:\Users\cleme\Desktop\master\project1-grp_e18\Script_python_annexes
collected 1 item

test_sort.py .                                                                                          [100%]

============================================== 1 passed in 4.67s ==============================================
```

FIGURE 4.2 – Résultat de l'éxécution de pytest

Chapitre 5

Bilan de projet

Contents

5.1 Bilan sur l'avancement du projet 22
5.1.1 Fonctionnalités implémentées 22
5.1.2 Fonctionnalités non réalisés 23
5.2 Bilan global du projet 23
5.2.1 Bilan de l'écriture du code 23
5.2.2 Bilan de la gestion de projet 24
5.2.3 Critique de l'éthique du projet 25
5.3 Bilan du projet par membre 25
5.3.1 ASSING Norman 25
5.3.2 COUCHEVELLOU Clément 26
5.3.3 COUVRAT–PAILLE Titouan 27
5.3.4 SEDEKI Khalyl 28

5.1 Bilan sur l'avancement du projet

5.1.1 Fonctionnalités implémentées

— Système de compte : création de compte, connexion, vérification des contraintes
— Messages audio : recording du message, téléchargement sur le serveur, possibilité de faire écouter le message à l'utilisateur, possibilité de donner un titre au message
— Débats : Création, affichage de tous les débats, affichage des propositions de chacun des débats

— Fonctionnalités contraintes par la connexion : “publier un commentaire ou une proposition” réservé aux utilisateurs connectés, “créer un débat” réservé aux administrateurs
— Possibilité de répondre à un commentaire
— Possibilité de citer une proposition pour en construire une nouvelle
— Statistiques : nombre de jetons utilisés sur chaque débat et chaque proposition, nombre de propositions qui ont cité une autre, nombre de commentaires à chaque proposition

5.1.2 Fonctionnalités non réalisés

— Email de confirmation de compte
— Faire un bouton/lien “mot de passe oublié”
— Conversion des fichiers audios en mp3
— Compression des fichiers pour optimiser l’espace occupé
— Lister toutes les propositions qui en cite une autre
— La page Paramètres/Réglages
— Le stockage des messages audio dans plusieurs dossiers pertinents
— La possibilité de fermer les débats
— L’archivage des débats
— Limiter le nombre de jetons de vote par jour
— Le système de “like” pour les commentaires
— Statistiques avancées : Utilisateurs les plus actifs, statistiques par jour/semaine/mois, ...
— Profils d’utilisateurs
— Faire des débats privés/publics (cette partie a commencé à être développée, mais il y a eu quelques pertes à cause de conflits git. Elle a donc été abandonnée, cependant il en reste des traces dans le fichier app.py, la base de donnée et dans les templates html)
— Ajouter une barre de recherche dans l’espace de débat afin

Ces fonctionnalités n’ont pas été implémentées par manque de temps ; aucune ne semble dépasser les capacités techniques de l’équipe

5.2 Bilan global du projet

5.2.1 Bilan de l’écriture du code

— Expérience commune

    - Progression dans différents langages de programmation tels que Flask, html, css, et SQL
    - Meilleure utilisation de Git
    - Découverte de l'importance de la gestion de projet

- Les points positifs

    - Une bien meilleure maîtrise de Flask, HTML, CSS, SQL
    - Première utilisation de pytest
    - Un résultat très satisfaisant par rapport à nos attentes initiales

- Les points négatifs

    - Un manque de rigueur dans l'écriture au début du projet
    - Adaptation à la manière de coder de chacun - pas toujours évident de travailler à partir de code écrit par un autre membre.
    - L'apprentissage des bases en JavaScript qui était inéluctable pour les messages audio

5.2.2 Bilan de la gestion de projet

- Les points positifs

    - Les parties définition et montage qui ont été plutôt bien gérées
    - La gestion de projet à été notre priorité → meilleure efficacité pour écrire le code après
    - Des échanges quotidiens qui ont empêchés l'effet tunnel
    - Beaucoup de ressources de gestion de projet, mises en place dès le début du projet, qui nous ont aidé à visualiser clairement, notre avancement et la direction du projet jusqu'à la fin.

- Les points négatifs

    - Un projet un peu trop ambitieux
    - Des comptes rendus parfois négligés
    - La gestion du temps difficile entre les partiels et les fêtes de fin d'année

5.2.3 Critique de l'éthique du projet

Le but de Kleps est de permettre aux habitants d'une commune de débattre sur des sujets donnés par leur mairie. Ainsi, le thème "Démocratie de proximité" est respecté. Cependant, nous sommes conscients que Kleps comporte des problèmes éthiques. Le premier concerne les personnes malentendantes ou muettes, les messages vocaux sont évidemment un frein au débat pour ces personnes là. Pour y remédier nous pourrions développer une interface "malentendants et muets" ou les messages vocaux seraient écrits, et ou leurs messages écrits seraient énoncés, soit par une IA soit par des modérateurs.

Le deuxième problème concerne l'anonymat sur le site. La question s'est posée à plusieurs reprises lors de réunion avec les membres de l'équipe : Doit-on être anonyme sur Kleps ? Aucune bonne réponse n'en est émergée. Si les gens avaient l'anonymat, Kleps deviendrait sûrement submergé de messages de haine etc... avec des utilisateurs se cachant derrière celui-ci. Cela dit, si l'anonymat n'était pas respecté, des conflits ouverts seraient susceptibles d'éclater avec des utilisateurs de différentes opinions. Les noms d'utilisateurs étant visibles par tous des violences physiques pourraient en découler. Pour l'instant, Kleps fonctionne avec des pseudonymes, libres aux utilisateurs de décliner leur identité ou non.

5.3 Bilan du projet par membre

5.3.1 ASSING Norman

- Les points positifs
  - Tous les membres de l'équipe étaient enthousiastes pour le projet. Chacun a pu apporter ses idées pour façonner l'application le mieux possible
  - Première expérience très marquante en tant que chef de projet
- Les difficultés rencontrées
  - Mauvaise gestion des comptes rendus
  - Volonté de terminer toutes les fonctionnalités quitte à passer la totalité de ses vacances sur le projet et au détriment de la partie gestion de projet
- L'expérience personnelle

"J'ai été trop investi sur ce projet et je ne sais pas vraiment si c'est une bonne chose. Je suis un bien meilleur développeur qu'il y a quelques semaines. Kleps est une application dont notre groupe peut être fier, surtout qu'elle est très proche de la version que j'avais en tête au départ. Cependant pour arriver à ce résultat, j'ai dédié l'intégralité de mes vacances à ce projet. Je travaillais sans relâche, sans prendre de jour de repos après les partiels ou lors des fêtes de fin d'année. J'ai dû faire des sacrifices sur ma vie personnelle pour une pression inexistante que je me suis fixé tout seul. Je ne sais pas si j'aurais un souvenir positif de cette expérience, mais à l'heure actuelle je suis juste content que cette aventure touche à sa fin."

— Les axes d'amélioration

  — Donner plus d'importance à la partie gestion de projet
  — Accepter de revoir à la baisse la quantité de fonctionnalités

5.3.2 COUCHEVELLOU Clément

— Les points positifs

  — Définition précise du projet dès le début grâce aux idées et présentation de Norman
  — Utilisation de nouvelles ressources
  — Travail en équipe efficace, bonne coopération dans le groupe
  — Plusieurs fonctionnalités assez poussées ont pu être ajoutées, malgré l'incertitude sur leur faisabilité au départ

— Les difficultés rencontrées

  — Manque de temps avec les partiels et les fêtes
  — Maitrise faible de Flask, HTML et CSS
  — Aucunes connaissances en JavaScript
  — Maîtrise difficile de git, majoritairement pour les branches, problèmes d'écrasement de fichiers

— L'expérience personnelle

"J'ai aimé le temps passé à faire du “problem solving” et à implémen-

ter de nouvelles fonctionnalités au projet, pour le voir se compléter chaque jour. Après avoir travaillé sur tous les aspects du projet, je me rends compte que la partie programmation est de loin celle que je préfère, en particulier dans ce projet la partie python/Flask. C'est notamment pour ces raisons que j'ai particulièrement apprécié le travail de groupe, qui m'a permis de passer plus de temps sur la programmation."

- Les axes d'amélioration
  - Mieux gérer mon temps, travailler peut-être moins régulièrement, mais plus efficacement, afin de pouvoir gérer plusieurs projets en même temps
  - Apprendre à mieux maîtriser git et tout particulièrement les branches

5.3.3 COUVRAT–PAILLE Titouan

- Les points positifs
  - Cohésion du groupe
  - Découvrir le potentiel du travail en commun
  - La satisfaction de produire quelque chose de concret qui a son utilité (potentiellement continuer de développer le projet pour le vendre ?)
  - Emploi du temps agréablement respecté / jalons atteints à 1 ou 2 jours près.
  - Découverte d'outils comme Trello super utiles.
- Les difficultés rencontrées
  - Tous les membres du groupe ne travaillaient pas à la même vitesse, peur de ne pas en faire assez
  - Manque de motivation dû au fait de devoir se former intégralement sur internet
  - Devoir synergiser avec tout le monde pour ne pas faire de travail inutile/que quelqu'un est déjà en train d'effectue
  - Pas réussi à faire fonctionner pytest sur mon pc
- L'expérience personnelle

"Je me suis formé sur flask html et css. Durant les réunions il fallait quelques fois prendre le devant pour rediriger les choses et être efficace. La gestion de projet ne m'était pas étrangère grâce à mon expérience de l'organisation du TN'Event dans le pôle Ressources Humaines. C'est plus facile de s'impliquer sur une direction choisie ensemble plus que sur une direction imposée."

- Les axes d'amélioration

  - travailler plus et plus régulièrement pour être plus efficace
  - Savoir mieux anticiper les charges de travail / estimer plus précisément les charges de travail.

5.3.4 SEDEKI Khalyl

- Les points positifs

  - Tous les membres de l'équipe étaient investis dans le projet. Chacun a su apporter ses idées et les adapter de façon pertinente à l'application.
  - C'est une première expérience très marquante en tant que membre d'un projet d'équipe.
  - Cela m'a permis de me rendre compte des éléments à prendre en compte lorsque tout ne se déroule pas comme prévu

- Les difficultés rencontrées

  - J'ai du reprendre tous les comptes rendus de réunion en fin de projet ce qui n'est pas évident
  - La gestion de mon temps de travail et de mon investissement qui demande à être peaufinée.
  - J'ai aussi dû faire face à des problématiques imprévisibles lors des vacances, (décès d'un membre de ma famille, je me suis retrouvé bloqué hors de chez moi sans mon matériel et j'ai eu une panne de mon ordinateur) et d'autres problématiques plus facile à prévoir comme les partiels

- L'expérience personnelle

  "Ce fut l'occasion pour moi d'en apprendre davantage sur Flask,

HTML et CSS, ainsi que sur mes capacités et mon efficacité dans le travail. Par ailleurs, sur le plan de la communication, j'ai appris à mieux gérer les informations et leur transmission . Je comprends dès à présent beaucoup mieux l'importance de la gestion de projet en général et l'utilité des jalons. Par ailleurs, mon expérience en tant que secrétaire du TN'event m'a aidé dans la réalisation des comptes-rendu de réunion."

— Les axes d'amélioration

  — Accorder plus de temps à la rédaction des comptes rendus. De plus je me rends compte qu'il ne faut pas remettre à plus tard et essayer d'avancer le plus rapidement possible sur les tâches simples pour avoir plus de temps à consacrer aux tâches complexes.
  — Sur le plan personnel, une communication plus rigoureuse et réactive sur les problèmes que je rencontre pourrait aider le reste de l'équipe à prendre mon relais ou avancer plus sereinement..